

# 01 — Arquitectura de Microservicios

---

**Proyecto:** Plataforma Libro de Clases Digital

**Asignatura:** DSY1106 — Desarrollo Fullstack III

**Evaluación:** Parcial N°3 — Encargo (base) · Actualizado para Examen 2026-07-13

**Equipo:** Cristian Monsalve / Héctor Olivares

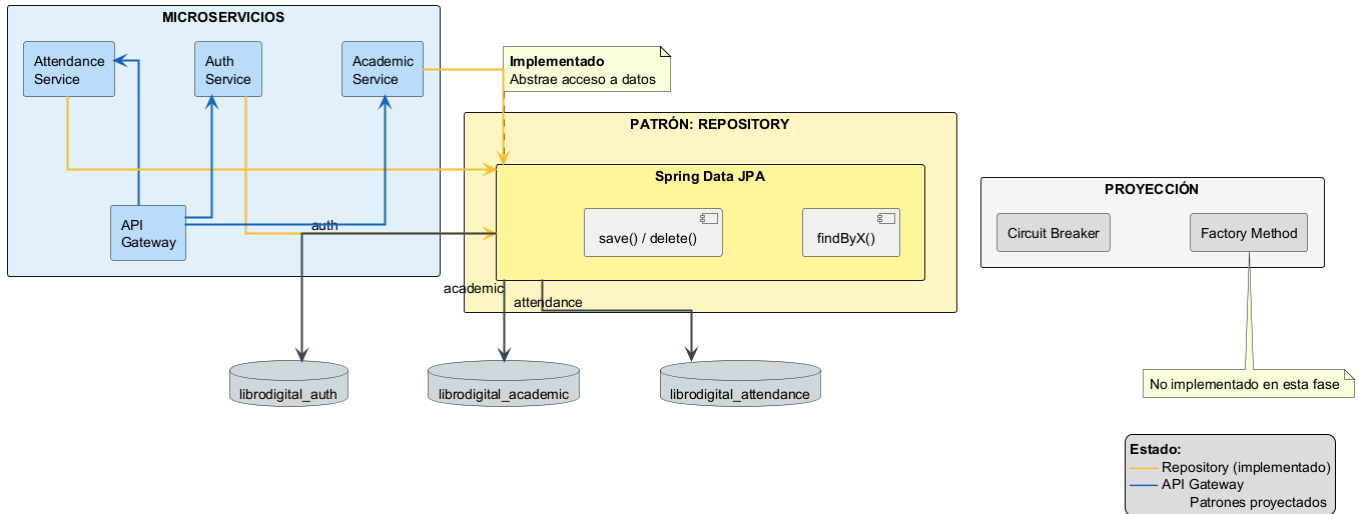
**Examen:** mejoras e innovación (alertas a apoderados) en `05_mejoras_e_innovacion_examen.md`.

## 1. Resumen ejecutivo

El sistema implementa una **arquitectura de microservicios** para la gestión académica de un colegio: autenticación centralizada, dominio académico (estudiantes, cursos, notas, **mensajería**), dominio de asistencia (sesiones, registros, anotaciones) y un **frontend por roles**. Todos los servicios backend se exponen al exterior a través de un **API Gateway**, y el **frontend React** consume únicamente ese punto de entrada.

La comunicación entre capas es **REST sobre HTTP/JSON**. Cada microservicio persiste en su **propia base PostgreSQL** (patrón *Database per Service*), sin claves foráneas entre bases distintas. Entre microservicios, cuando hace falta orquestación ligera (ej. avisar al apoderado al guardar asistencia), se usa **llamada REST** (`attendanceService` → `academicService`).

## 2. Diagrama de arquitectura



Fuente editable: `infraestructura/diagrams/architecture_patterns_simple.puml`

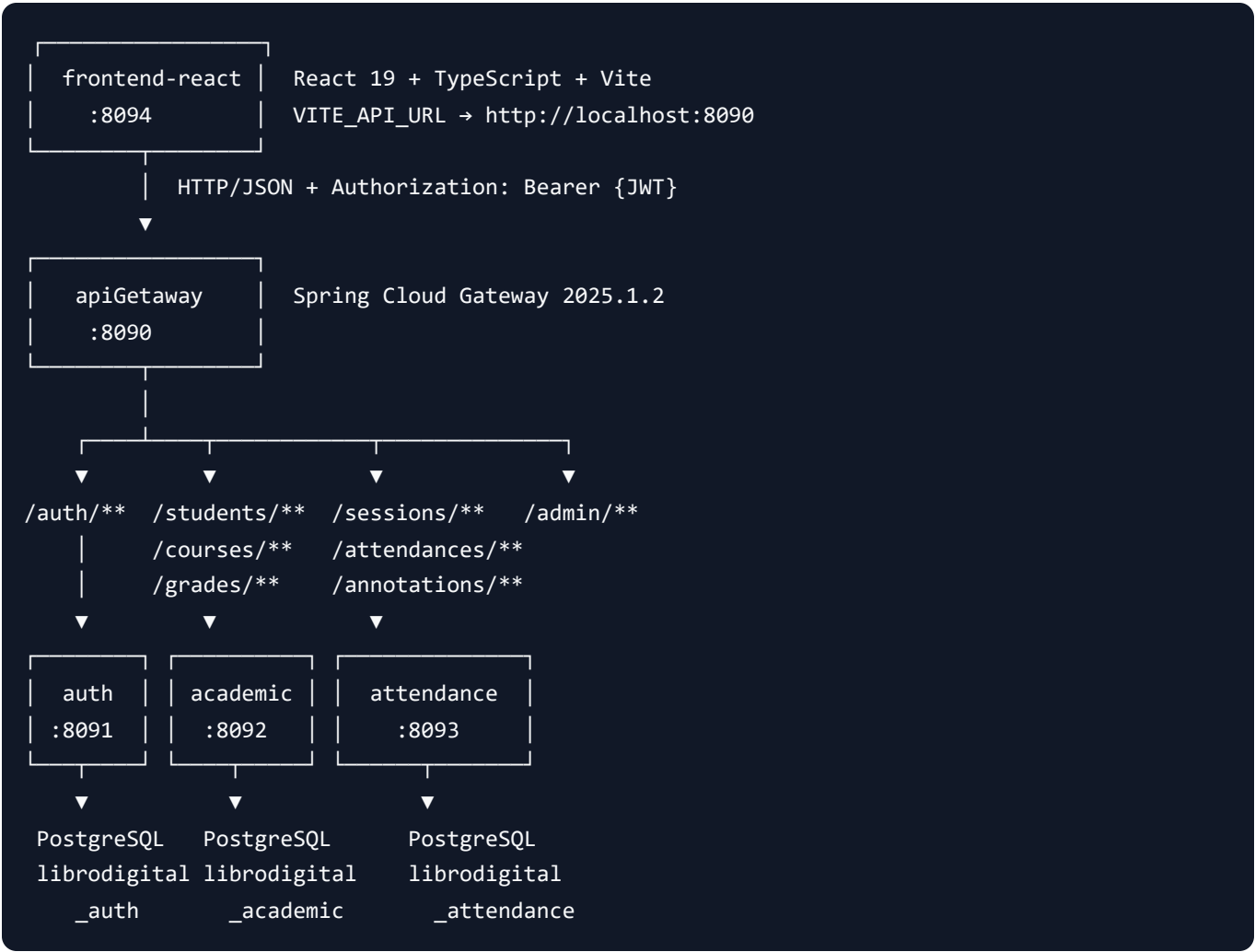
El diagrama muestra:

- **Cliente web** ( `frontend-react` , puerto 8094)
- **API Gateway** ( `apiGateway` , puerto 8090) como único punto de entrada
- **Tres microservicios** backend con sus bases de datos dedicadas
- Patrones aplicados: API Gateway, Database per Service, Repository, DTO, Service Layer, MVC

### 3. Componentes del sistema

| Componente         | Carpeta                         | Puerto | Responsabilidad                                                                           |
|--------------------|---------------------------------|--------|-------------------------------------------------------------------------------------------|
| API Gateway        | <code>apiGateway/</code>        | 8090   | Enrutamiento, CORS, proxy hacia microservicios                                            |
| Auth Service       | <code>authService/</code>       | 8091   | Login, JWT, usuarios, roles (RBAC)                                                        |
| Academic Service   | <code>academicService/</code>   | 8092   | Estudiantes, docentes, cursos, matrículas, evaluaciones, notas, <b>mensajes / alertas</b> |
| Attendance Service | <code>attendanceService/</code> | 8093   | Sesiones de clase, asistencia, anotaciones (+ notify a academic)                          |
| Frontend React     | <code>frontend-react/</code>    | 8094   | UI por rol (Dirección, Oficina, docente, apoderado, estudiante)                           |

## 4. Flujo de integración Frontend ↔ Backend ↔ Persistencia



### Rutas del Gateway (extracto)

| Prefijo                                                     | Destino               | Servicio                              |
|-------------------------------------------------------------|-----------------------|---------------------------------------|
| /auth/**                                                    | http://localhost:8091 | authService                           |
| /admin/**                                                   | http://localhost:8091 | authService                           |
| /students/**, /courses/**, /teachers/**, /subjects/**       | http://localhost:8092 | academicService                       |
| /enrollments/**, /evaluations/**, /grades/**, /guardians/** | http://localhost:8092 | academicService                       |
| /messages/**                                                | http://localhost:8092 | academicService (mensajería + notify) |

| Prefijo                                             | Destino               | Servicio          |
|-----------------------------------------------------|-----------------------|-------------------|
| /sessions/** , /attendances/** ,<br>/annotations/** | http://localhost:8093 | attendanceService |

El frontend **no** invoca directamente los puertos 8091–8093 en desarrollo; siempre usa el gateway en **8090**, lo que desacopla la UI de la topología interna.

---

## 5. API REST

La especificación de endpoints está documentada en:

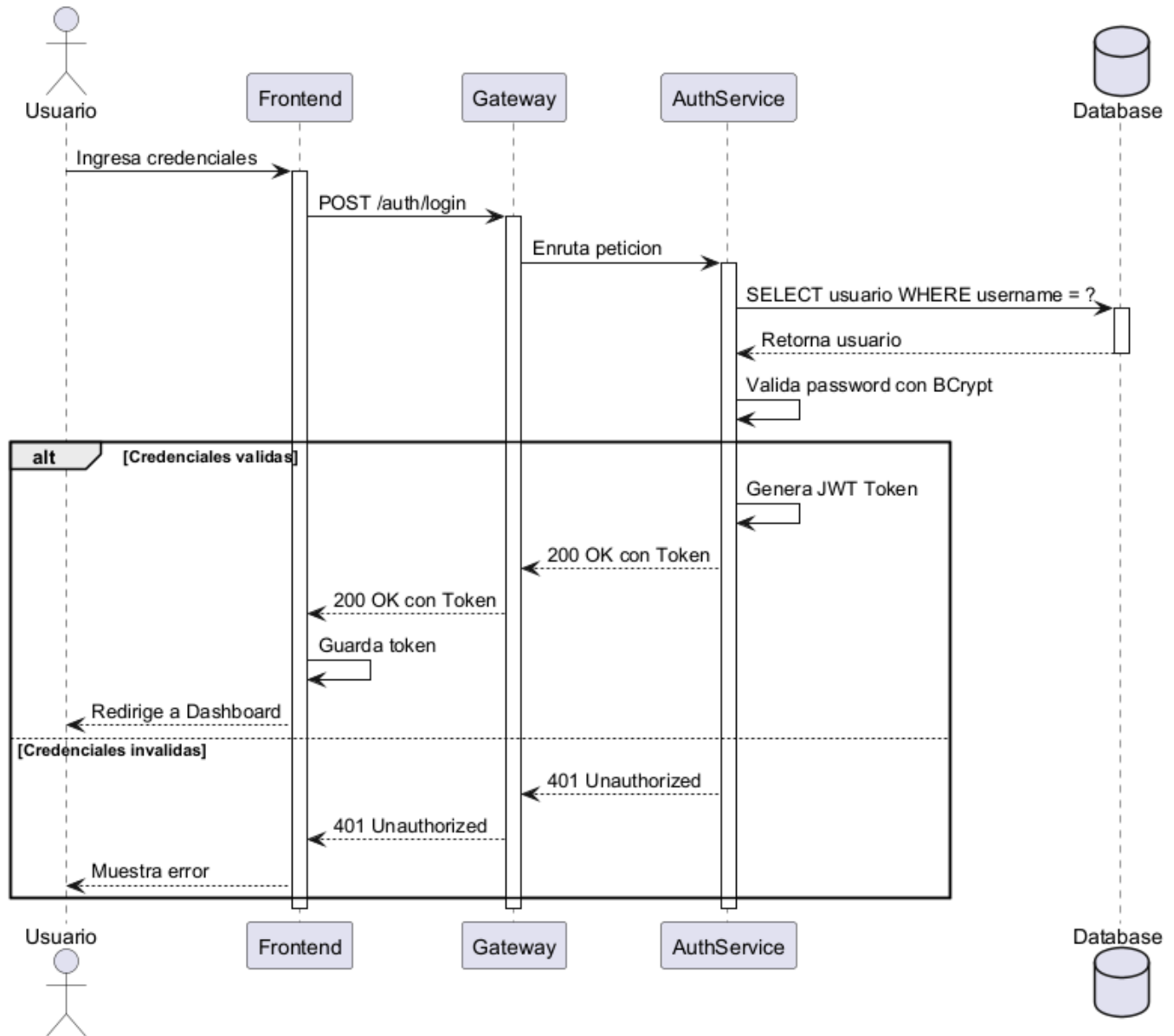
- **Colección Postman:** `infraestructura/postman/Libro_Digital.postman_collection.json`
- **Guía de uso:** `infraestructura/postman/README.md`

Flujo típico de prueba:

1. POST `/auth/login` → obtiene `accessToken`
  2. Peticiones subsiguientes con header `Authorization: Bearer {token}`
  3. Operaciones CRUD en dominios academic y attendance según rol del usuario
- 

## 6. Seguridad transversal

## Flujo de Autenticacion - Login



1. El usuario envía credenciales a `POST /auth/login`.
2. `authService` valida con **BCrypt** y emite **JWT** (`accessToken` + `refreshToken`).
3. El **gateway** y cada microservicio validan el token en rutas protegidas.
4. **RBAC** con roles efectivos: `SUPER_ADMINISTRADOR` (Dirección), `ADMINISTRADOR` / `ADMINISTRATIVO` (Oficina), `DOCENTE`, `APODERADO`, `ESTUDIANTE`.
5. Registro público deshabilitado; provisión vía `POST /admin/users`.

Usuarios demo (password `test1234`): `admin_colegio`, `admin_oficina`, `prof_castillo`, `apoderado_demo`, `estudiante_demo`.

## 7. Stack tecnológico

| Capa            | Tecnologías                                                         |
|-----------------|---------------------------------------------------------------------|
| Backend         | Java 21, Spring Boot 4.1.0, Spring Security, Spring Data JPA, Maven |
| Gateway         | Spring Cloud Gateway 2025.1.2                                       |
| Frontend        | React 19, TypeScript, Vite 8, Tailwind CSS, React Router            |
| Base de datos   | PostgreSQL 16+ (una instancia, tres bases lógicas)                  |
| Testing backend | JUnit 5, Mockito, Spring Boot Test, JaCoCo 0.8.12                   |
| API testing     | Postman Collection                                                  |

## 8. Patrones de diseño implementados

| Patrón               | Aplicación en el proyecto                                              |
|----------------------|------------------------------------------------------------------------|
| API Gateway          | <code>apiGateway</code> centraliza rutas y simplifica el cliente       |
| Database per Service | Tres bases PostgreSQL independientes                                   |
| Repository           | Interfaces <code>JpaRepository</code> por entidad                      |
| DTO                  | Objetos de transferencia separados de entidades JPA                    |
| Service Layer        | Lógica de negocio en <code>*ServiceImpl</code> , controllers delegados |
| MVC                  | Controllers REST + modelos JPA + vistas React                          |

Detalle ampliado: `infraestructura/docs/patrones.md`

## 9. Orden de arranque recomendado

- PostgreSQL (bases creadas con `infraestructura/ddl/00_create_databases.sql`)
- `authService` → puerto 8091
- `academicService` → puerto 8092
- `attendanceService` → puerto 8093
- `apiGateway` → puerto 8090
- `frontend-react` → puerto 8094 ( `npm run dev` )

## 10. Diagramas complementarios

| Diagrama          | Ubicación                                           | Contenido                                                               |
|-------------------|-----------------------------------------------------|-------------------------------------------------------------------------|
| ER Auth           | infraestructura/diagrams/auth_er_model.png          | Tablas <code>users</code> , <code>role</code> , <code>user_roles</code> |
| ER Academic       | infraestructura/diagrams/academic_er_model.png      | Catálogos + entidades académicas (3FN)                                  |
| ER Attendance     | infraestructura/diagrams/attendance_er_model.png    | Sesiones, asistencia, anotaciones                                       |
| Autorización RBAC | infraestructura/diagrams/security_authorization.png | Control de acceso por rol                                               |

## 11. Evolución post-EP3 (examen)

Sin cambiar el patrón de microservicios, se incorporó:

- Módulo de **mensajería** en `academicService`
- **Orquestación ligera** attendance → academic para alertas de asistencia
- Alertas al **crear evaluaciones** (mismo dominio academic)
- UI con confirmaciones y portals por rol más claros

Detalle: `05_mejoras_e_innovacion_examen.md`.

## 12. Conclusión

La arquitectura cumple los requisitos del encargo: **microservicios desacoplados**, **integración frontend-backend vía API REST**, **persistencia independiente por servicio** y **punto de entrada único** (gateway). Para el examen, esa base sostiene además una **innovación de comunicación en tiempo casi real** hacia apoderados, sin romper el aislamiento de bases de datos.